

LAB 1

Q1 – Outer vs Inner:

Outer = customer (Index Scan, 1 row). Inner = rental (Seq Scan, filtered to 32 rows).

Q2 – Why Nested Loop, not Hash Join?

customer returns only 1 row after filtering → looping just once is cheap, even though rental needs a full Seq Scan. Hash Join would still need to scan rental fully to build the hash table, plus extra overhead — not worth it for only 1 outer row.

Q3 – Why loops = 1?

Because the outer table (customer) has exactly 1 matching row, so the inner scan (rental) only runs once.

The screenshot shows a PostgreSQL IDE interface. On the left is the 'Object Explorer' showing the database structure. The main window displays a SQL query and its execution plan.

Query:

```
1 EXPLAIN ( BUFFERS )
2 SELECT * FROM customer WHERE last_name = ' Kelly' ;
3
4
5 EXPLAIN ( ANALYZE , BUFFERS )
6 SELECT c . first_name , c . last_name , r . rental_date
7 FROM customer c
8 JOIN rental r ON c . customer_id = r . customer_id
9 WHERE c . customer_id = 1 ;
```

Query Plan:

Step	Plan
1	Nested Loop (cost=0.28..359.16 rows=32 width=21) (actual time=0.859..4.186 rows=32 loops=1)
2	Buffers: shared read=153
3	-> Index Scan using customer_pkey on customer c (cost=0.28..8.29 rows=1 width=17) (actual time=0.649..0.652 rows=1 loops=1)
4	Index Cond: (customer_id = 1)
5	Buffers: shared read=3
6	-> Seq Scan on rental r (cost=0.00..350.55 rows=32 width=10) (actual time=0.204..3.522 rows=32 loops=1)
7	Filter: (customer_id = 1)
8	Rows Removed by Filter: 16012
9	Buffers: shared read=150
10	Planning:
11	Buffers: shared hit=125 read=11 dirtied=2

Total rows: 13 Query complete 00:00:00.055

LAB 2

The screenshot shows a PostgreSQL IDE interface. On the left is the 'Object Explorer' showing the database structure. The main pane displays a SQL query and its execution plan.

Query:

```

1 EXPLAIN ( BUFFERS )
2 SELECT * FROM customer WHERE last_name = ' Kelly' ;
3
4
5 EXPLAIN ( ANALYZE , BUFFERS )
6 SELECT c . first_name , c . last_name , r . rental_date
7 FROM customer c
8 JOIN rental r ON c . customer_id = r . customer_id
9 WHERE c . customer_id = 1 ;
10
11 EXPLAIN ( ANALYZE , COSTS , VERBOSE , BUFFERS )
12 SELECT f.title , l.name as language
13 FROM film f
14 JOIN language l ON f.language_id = l.language_id;

```

Query Plan:

Step	Operation	Costs	Buffers
1	Hash Join	(cost=1.14..103.62 rows=1000 width=99) (actual time=0.380..2.588 rows=1000 loops=1)	
2	Output: f.title, l.name		
3	Inner Unique: true		
4	Hash Cond: (f.language_id = l.language_id)		
5	Buffers: shared read=89		
6	-> Seq Scan on public.film f	(cost=0.00..98.00 rows=1000 width=17) (actual time=0.174..2.104 rows=1000 loops=1)	
7	Output: f.film_id, f.title, f.description, f.release_year, f.language_id, f.rental_duration, f.rental_rate, f.length, f.replacement_cost, f.rating, f.last_update, f.special_features, ...		
8	Buffers: shared read=88		
9	-> Hash	(cost=1.06..1.06 rows=6 width=88) (actual time=0.193..0.194 rows=6 loops=1)	
10	Output: l.name, l.language_id		
11	Buckets: 1024 Batches: 1 Memory Usage: 9kB		

Total rows: 19 Query complete 00:00:00.087

Q1 Build vs Probe:

Build = language (smaller table, hashed into memory). Probe = film (larger table, scanned and matched against the hash).

Q2 Memory Check:

Look at the Hash node's Memory Usage line — should be very small (a few KB), since language has only a handful of rows.

Q3 Hypothesis (work_mem = 64kB):

Likely no change, language is tiny, so 64kB is still enough to fit in memory (Batches: 1). To actually trigger a disk spill (Batches > 1), you'd need a bigger table on the build side or an even smaller work_mem.

LAB 3

The screenshot shows the PostgreSQL Query Editor interface. On the left, the Object Explorer displays the database structure, including the 'customer' table. The main query window contains the following SQL code:

```

8 JOIN rental r ON c . customer_id = r . customer_id
9 WHERE c . customer_id = 1 ;
10
11 EXPLAIN (ANALYZE, COSTS, VERBOSE, BUFFERS)
12 SELECT f.title, l.name as language
13 FROM film f
14 JOIN language l ON f.language_id = l.language_id;
15
16 SET enable_hashjoin = OFF;
17 SET enable_nestloop = OFF;
18
19 EXPLAIN (ANALYZE, BUFFERS)
20 SELECT f.film_id, f.title, i.inventory_id
21 FROM film f
22 JOIN inventory i ON f.film_id = i.film_id
23 ORDER BY f.film_id;
24
25

```

The Query Plan output shows the following steps:

- 1 Merge Join (cost=349.64..547.91 rows=4581 width=23) (actual time=2.672..5.359 rows=4581 loops=1)
- 2 Merge Cond: (f.film_id = i.film_id)
- 3 Buffers: shared hit=109
- 4 -> Index Scan using film_pkey on film f (cost=0.28..127.33 rows=1000 width=19) (actual time=0.005..0.396 rows=1000 loops=1)
- 5 Buffers: shared hit=84
- 6 -> Sort (cost=349.37..360.82 rows=4581 width=6) (actual time=2.662..3.151 rows=4581 loops=1)
- 7 Sort Key: i.film_id
- 8 Sort Method: quicksort Memory: 336kB
- 9 Buffers: shared hit=25
- 10 -> Seq Scan on inventory i (cost=0.00..70.81 rows=4581 width=6) (actual time=0.005..1.260 rows=4581 loops=1)
- 11 Buffers: shared hit=25

The status bar indicates: Total rows: 15 Query complete 00:00:00.053. A green message box says: "Successfully run. Total query runtime: 53 msec. 15 rows affected."

The screenshot shows the PostgreSQL Query Editor interface after a RESET. The SQL code is as follows:

```

13 FROM film f
14 JOIN language l ON f.language_id = l.language_id;
15
16 SET enable_hashjoin = OFF;
17 SET enable_nestloop = OFF;
18
19 EXPLAIN (ANALYZE, BUFFERS)
20 SELECT f.film_id, f.title, i.inventory_id
21 FROM film f
22 JOIN inventory i ON f.film_id = i.film_id
23 ORDER BY f.film_id;
24
25 RESET ALL;
26
27 EXPLAIN (ANALYZE, BUFFERS)
28 SELECT f.film_id, f.title, i.inventory_id
29 FROM film f
30 JOIN inventory i ON f.film_id = i.film_id;
31

```

The Query Plan output shows the following steps:

- 1 Hash Join (cost=110.50..193.39 rows=4581 width=23) (actual time=0.820..3.370 rows=4581 loops=1)
- 2 Hash Cond: (i.film_id = f.film_id)
- 3 Buffers: shared hit=113
- 4 -> Seq Scan on inventory i (cost=0.00..70.81 rows=4581 width=6) (actual time=0.009..0.636 rows=4581 loops=1)
- 5 Buffers: shared hit=25
- 6 -> Hash (cost=98.00..98.00 rows=1000 width=19) (actual time=0.802..0.803 rows=1000 loops=1)
- 7 Buckets: 1024 Batches: 1 Memory Usage: 59kB
- 8 Buffers: shared hit=88
- 9 -> Seq Scan on film f (cost=0.00..98.00 rows=1000 width=19) (actual time=0.007..0.425 rows=1000 loops=1)
- 10 Buffers: shared hit=88
- 11 Planning:

The status bar indicates: Total rows: 14 Query complete 00:00:00.134. The bottom right corner shows: LF Ln 30, Col 43.

Q1 Is there a Sort node?

Yes, but only on one side. film used an Index Scan (already sorted, no Sort needed). inventory had no usable index, so it needed a Sort node (quicksort, 336kB) before merging.

Q2 After RESET + removing ORDER BY, does it switch back to Hash Join?

Yes. Without the forced settings and without ORDER BY, Postgres switched to Hash Join —

faster (0.82–3.37ms vs 2.67–5.36ms) since film fit easily in a small hash table (59kB) and no sort was needed.